

APÊNDICE A – MÓDULO 1: CIRCUITOS BÁSICOS ENVOLVENDO LED, RESISTORES E BOTÕES

A.1 Atividade 1: Acionamento de LED em Porta Digital

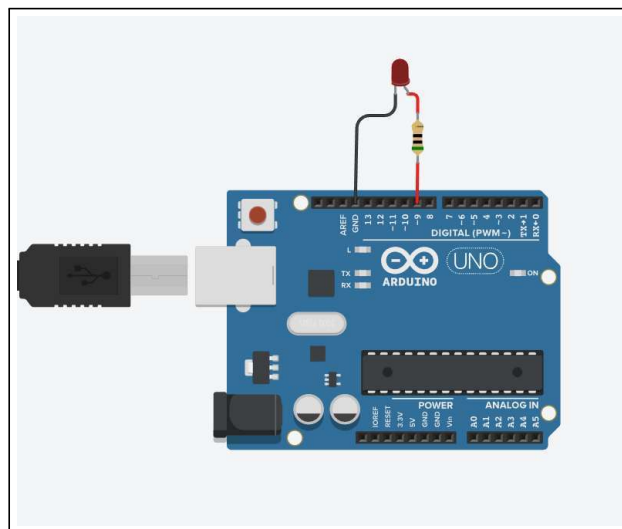
Conteúdo: Circuito básico com LED, resistor e porta digital.

Objetivo: Desenvolver a compreensão sobre acionamento digital, polaridade do LED e fluxo de corrente em portas digitais do Arduino, utilizando simulação computacional no *Tinkercad*.

Atividade Computacional – Etapas

1. Acesse a plataforma *Tinkercad* e crie um novo circuito.
2. Transfira para o ambiente virtual o esquema de ligação da Figura 10, que consiste em:
 - LED conectado à porta digital D9;
 - Resistor de 220 Ω em série;
 - Retorno do LED ao GND.
3. Após revisar as conexões, programe o Arduino conforme as questões A, B e C.

Figura 10 – Esquema de ligação – LED simples.



Fonte: Elaborada pelo autor.

Questão A – Porta D9 em nível lógico LOW

Após montar o circuito da Figura 10 no *Tinkercad*, programe a porta digital D9 em nível lógico LOW (0V).

Código:

Código-fonte 1 – Código para nível LOW na porta D9.

```
1 void setup() {  
2   pinMode(9, OUTPUT);  
3   digitalWrite(9, LOW);  
4 }  
5 void loop() {}
```

Prever

Com a porta D9 configurada em LOW, o LED irá acender?

- () Sim
- () Não

Observar e explicar

Após executar a simulação: Descreva o comportamento do LED e explique por que ele se comportou dessa forma, relacionando o nível lógico LOW e a polaridade do LED.

Questão B – Porta D9 em nível lógico HIGH

Agora, altere o código para configurar a porta D9 em nível lógico HIGH (5V).

Código:

Código-fonte 2 – Código para nível HIGH na porta D9.

```
1 void setup() {  
2   pinMode(9, OUTPUT);
```

```
3   digitalWrite(9, HIGH);  
4 }  
5 void loop() {}
```

Prever

Com a porta D9 configurada em HIGH, o LED irá acender?

- () Sim
- () Não

Observar e explicar

Após iniciar a simulação: Descreva o comportamento observado do LED e explique por que o nível HIGH altera seu funcionamento em comparação ao LOW.

Questão C – LED piscando com delay

Agora, modifique o programa para alternar o estado do LED a cada 1 segundo.

Código:

Código-fonte 3 – Código para piscar o LED.

```
1 void setup() {  
2   pinMode(9, OUTPUT);  
3 }  
4 void loop() {  
5   digitalWrite(9, HIGH);  
6   delay(1000);  
7   digitalWrite(9, LOW);  
8   delay(1000);  
9 }
```

Prever (marcar Sim/Não)

Com esse código, o LED irá piscar (acender e apagar repetidamente)?

- () Sim
- () Não

Observar e explicar

Após executar a simulação: Descreva o comportamento do LED e explique como os comandos `digitalWrite()` e `delay()` controlam o ciclo de execução.

A.2 Atividade 2: Leitura Digital com Botão

Conteúdo: Leitura digital (`digitalRead()`), acionamento condicional e uso do botão como entrada.

Objetivo: Compreender como o Arduino interpreta o estado de um botão e como esse sinal pode ser utilizado para controlar um atuador simples (LED) no ambiente virtual *Tinkercad*.

Atividade Computacional – Etapas

1. Abra a plataforma *Tinkercad* e crie um novo circuito.
2. Reproduza o esquema de ligação da Figura 11, contendo:
 - Um botão conectado ao pino digital D7;
 - Um resistor de $10k\Omega$ entre D7 e GND (*pull-down*);
 - Um LED conectado ao pino digital D13, com resistor de 220Ω em série.
3. Após revisar as ligações, programe o Arduino conforme as questões A, B e C.

Questão A – Leitura e Acionamento Direto

Código:

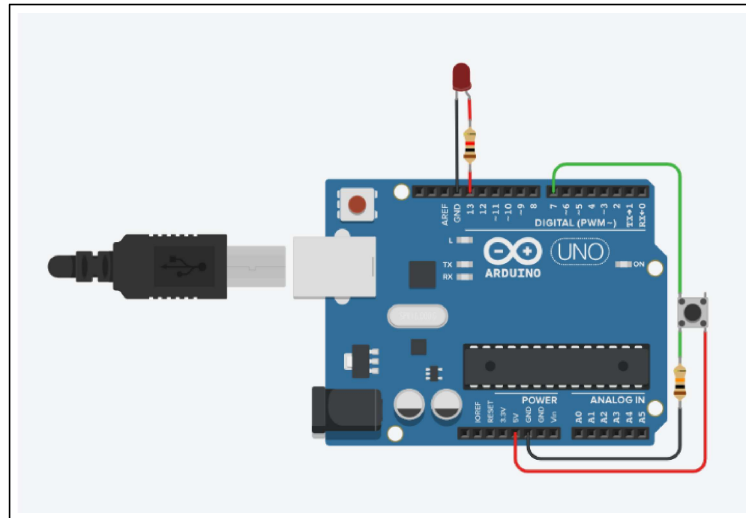
Código-fonte 4 – Leitura direta do botão para acionar o LED.

```

1  int ledPin = 13;
2  int inPin = 7;
3

```

Figura 11 – Esquema de ligação com botão e LED.



Fonte: Elaborada pelo autor.

```

4 void setup() {
5     pinMode(ledPin, OUTPUT);
6     pinMode(inPin, INPUT);
7 }
8
9 void loop() {
10    digitalWrite(ledPin, digitalRead(inPin));
11 }

```

Prever

Ao pressionar o botão conectado ao pino 7, o LED no pino 13 irá acender?

- ☐ Sim
- ☐ Não

Observar e explicar

Após executar a simulação: Descreva como o LED se comportou e explique como a leitura digital do botão determina o estado enviado ao LED.

Questão B – Invertendo a Lógica

Altere o programa para inverter o comportamento do LED.

Código:

Código-fonte 5 – Lógica invertida para acionamento do LED.

```
1 void setup() {}  
2  
3 void loop() {  
4     int leitura = digitalRead(7);  
5     digitalWrite(13, !leitura);  
6 }
```

Prever

Com essa nova lógica, o LED acenderá quando o botão não estiver pressionado (estado solto)?

- () Sim
- () Não

Observar e explicar

Descreva como o LED se comportou ao pressionar e soltar o botão. Explique como o operador lógico ! (negação) altera a relação entre o estado lido no botão e o estado aplicado ao LED.

Questão C – Botão como “Chave de Estado” (Toggle)

Agora, faça o LED alternar entre ligado e desligado a cada aperto do botão.

Código sugerido:

Código-fonte 6 – Botão atuando como alternador (toggle).

```
1 int estadoLED = 0;
2 int anterior = 0;
3
4 void setup() {
5     pinMode(7, INPUT);
6     pinMode(13, OUTPUT);
7 }
8
9 void loop() {
10     int atual = digitalRead(7);
11     if (atual == HIGH && anterior == LOW) {
12         estadoLED = !estadoLED;
13         digitalWrite(13, estadoLED);
14         delay(250);
15     }
16     anterior = atual;
17 }
```

Prever

Com esse código, o LED permanecerá no novo estado mesmo após soltar o botão?

- () Sim
- () Não

Observar e explicar

Após a simulação: Descreva o comportamento do LED a cada apertado do botão. Explique como a lógica de detecção de borda transforma o botão em um interruptor que alterna estados (*toggle*).

A.3 Atividade 3: Acionamento Sequencial de LEDs

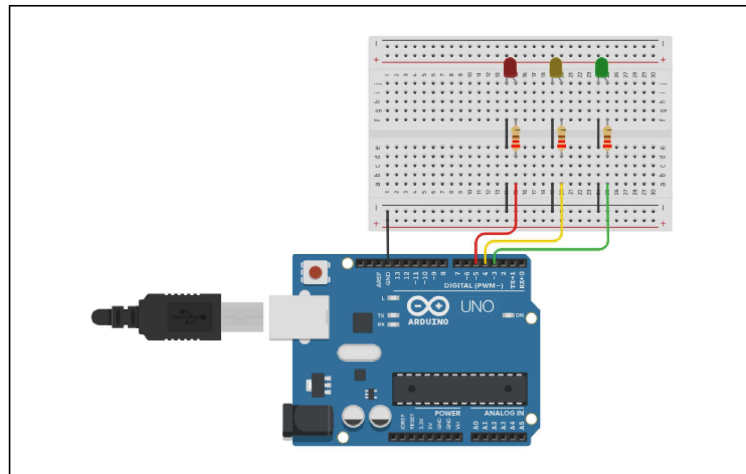
Conteúdo: Controle múltiplo de saídas digitais e lógica sequencial.

Objetivo: Compreender como o Arduino pode controlar diversos LEDs independentemente e criar sequências de acionamento.

Atividade Computacional – Etapas

1. Abra o *Tinkercad* e monte o circuito da Figura 12, contendo três LEDs conectados aos pinos D3, D4 e D5, cada um com resistor de 220Ω .
2. Após revisar as conexões, programe o Arduino conforme as questões A, B e C.

Figura 12 – Esquema de ligação com três LEDs sequenciais.



Fonte: Elaborada pelo autor.

Questão A – Acionamento Independente

Código:

Código-fonte 7 – Acionamento independente do LED no pino 3.

```

1 void setup() {
2   pinMode(3, OUTPUT);
3   pinMode(4, OUTPUT);
4   pinMode(5, OUTPUT);
5
6   digitalWrite(3, HIGH);
7   digitalWrite(4, LOW);
8   digitalWrite(5, LOW);

```



```

9  }
10
11 void loop() {}

```

Prever

Com esse código, apenas o LED do pino 3 irá acender?

- ☐ Sim
- ☐ Não

Observar e explicar

Após simular: Descreva quais LEDs acenderam e explique por que somente a porta configurada em nível HIGH ativou seu LED, enquanto as demais permaneceram desligadas.

Questão B – Dois LEDs Simultâneos

Código:

Código-fonte 8 – Acionamento simultâneo de dois LEDs.

```

1 void setup() {
2   pinMode(3, OUTPUT);
3   pinMode(4, OUTPUT);
4   pinMode(5, OUTPUT);
5
6   digitalWrite(3, LOW);
7   digitalWrite(4, HIGH);
8   digitalWrite(5, HIGH);
9 }
10
11 void loop() {}

```

Prever

Com esse código, dois LEDs irão acender ao mesmo tempo (pinos 4 e 5)?

- ☐ Sim
- ☐ Não

Observar e explicar

Após simular: Explique o comportamento dos LEDs e descreva como o Arduino consegue ativar múltiplas portas digitais simultaneamente sem interferência entre elas.

Questão C – Sequência de Piscar***Código:*****Código-fonte 9 – Sequência de acionamento dos LEDs.**

```
1 void setup() {  
2     pinMode(3, OUTPUT);  
3     pinMode(4, OUTPUT);  
4     pinMode(5, OUTPUT);  
5 }  
6  
7 void loop() {  
8     digitalWrite(3, HIGH); delay(300);  
9     digitalWrite(3, LOW);  
10  
11     digitalWrite(4, HIGH); delay(300);  
12     digitalWrite(4, LOW);  
13  
14     digitalWrite(5, HIGH); delay(300);  
15     digitalWrite(5, LOW);  
16 }
```

Prever

Com esse código, os três LEDs piscarão em sequência (um após o outro)?

- () Sim
- () Não

Observar e explicar

Após a simulação: Descreva a ordem em que os LEDs acenderam e explique como o `delay()` e o comando `digitalWrite()` determinam a sequência temporal.

A.4 Atividade 4: Sensores (LDR) e Leitura Analógica

Conteúdo: Leitura analógica (`analogRead()`), divisor resistivo com LDR e análise de variação de tensão.

Objetivo: Identificar como a resistência da LDR altera a leitura analógica e como essa leitura pode ser usada para acionar LEDs.

Atividade Computacional – Etapas

1. Monte o circuito da Figura 13, contendo:
 - LDR formando divisor resistivo com resistor de $10k\Omega$;
 - Saída do divisor conectada ao pino A0;
 - LED conectado ao pino digital D11 (PWM).
2. Após revisar a montagem, programe o Arduino conforme as questões A, B e C.

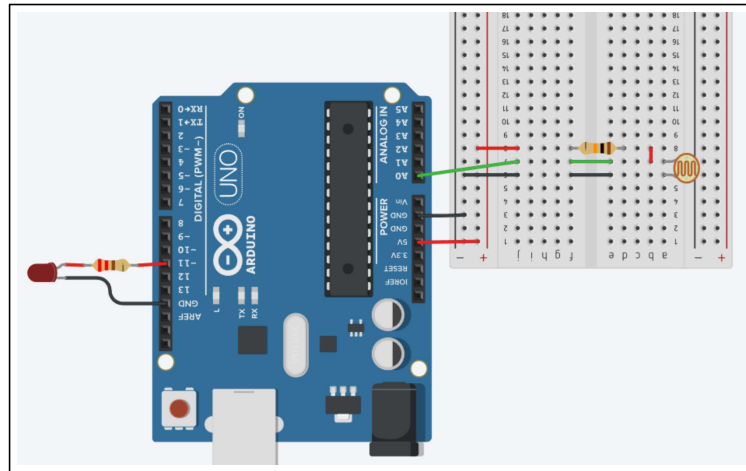
Questão A – Leitura do Sensor**Código:****Código-fonte 10 – Leitura analógica da LDR e impressão no Monitor Serial.**

```

1 void setup() {
2     Serial.begin(9600);
3 }
4

```

Figura 13 – Leitura analógica utilizando LDR como sensor de luminosidade.



Fonte: Elaborada pelo autor.

```

5 void loop() {
6     int valor = analogRead(A0);
7     Serial.println(valor);
8     delay(300);
9 }

```

Prever

Ao rodar o código inicial, o LED do pino 11 acenderá imediatamente?

- () Sim
- () Não

Observar e explicar

Após executar a simulação: Descreva como os valores exibidos no Monitor Serial variaram e explique por que esses valores mudam quando a luminosidade sobre a LDR aumenta ou diminui.

Questão B – Controle de Brilho

Código:

Código-fonte 11 – Controle de brilho proporcional ao valor lido na LDR.

```

1 void setup() {
2     pinMode(11, OUTPUT);
3 }
4
5 void loop() {
6     int val = analogRead(A0);
7     analogWrite(11, val/4);
8 }

```

Prever

Com este código, o brilho do LED em D11 irá variar proporcionalmente à luminosidade medida pela LDR?

- () Sim
- () Não

Observar e explicar

Após executar a simulação: Descreva como o brilho do LED mudou ao variar a luminosidade sobre a LDR. Explique a relação entre:

- O valor obtido com `analogRead()` (0 a 1023);
- O mapeamento para `analogWrite()` realizado pela divisão por 4 (0 a 255);
- O efeito final no PWM que controla o brilho do LED.

Comente também sobre limites perceptíveis, como valores mínimos que ainda produzem luminosidade visível.

Questão C – LED como Alarme de Sombreamento

Código:

Código-fonte 12 – Uso da LDR como alarme por limiar de luminosidade.

```

1 void setup() {

```

```
2   pinMode(11, OUTPUT);  
3 }  
4  
5 void loop() {  
6     int val = analogRead(A0);  
7     if (val < 400) digitalWrite(11, HIGH);  
8     else digitalWrite(11, LOW);  
9 }
```

Prever

Com este código, o LED em D11 acenderá quando a LDR estiver coberta (valor abaixo do limiar de 400)?

- () Sim
- () Não

Observar e explicar

Após simular: Descreva se o LED acendeu nas situações de pouca luz e explique por que um limiar é utilizado. Comente sobre:

- Como escolher o valor do limiar (por exemplo, observando leituras do `analogRead()`);
- Possíveis efeitos de ruído e flutuação da leitura;
- Necessidade de filtragem ou técnicas de estabilização.

A.5 Atividade 5: Projeto Final – Semáforo com Temporização

Conteúdo: Estruturas sequenciais, temporização e lógica de sistema.

Objetivo: Programar um semáforo básico utilizando três LEDs e controle de tempo, integrando os conceitos desenvolvidos nos guias anteriores.

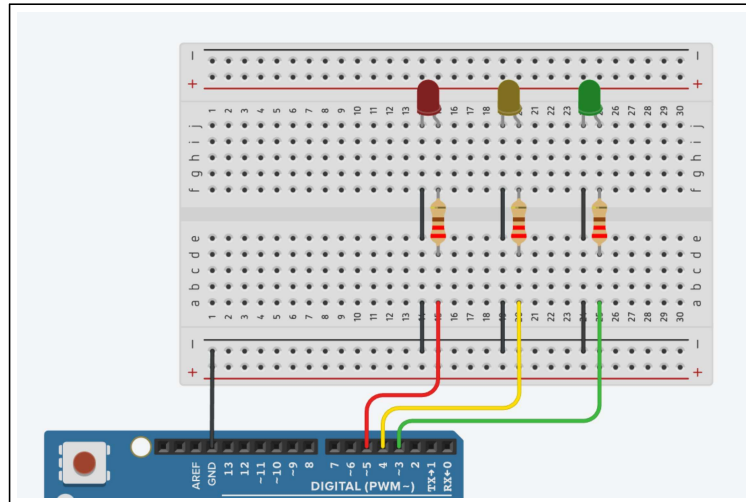
Atividade Computacional – Etapas

1. Monte o circuito da Figura 14, contendo:

- LED verde conectado ao pino D3;

- LED amarelo conectado ao pino D4;
 - LED vermelho conectado ao pino D5.
2. Após revisar todas as conexões, programe conforme as questões A, B e C.

Figura 14 – Semáforo simples com três LEDs.



Fonte: Elaborada pelo autor.

Questão A – Sequência Padrão

Código utilizado:

Código-fonte 13 – Sequência padrão do semáforo.

```

1 void setup() {
2     pinMode(3, OUTPUT);
3     pinMode(4, OUTPUT);
4     pinMode(5, OUTPUT);
5 }
6
7 void loop() {
8     digitalWrite(3, HIGH); delay(4000);
9     digitalWrite(3, LOW);
10
11     digitalWrite(4, HIGH); delay(1500);
12     digitalWrite(4, LOW);

```

```
13  
14   digitalWrite(5, HIGH); delay(4000);  
15   digitalWrite(5, LOW);  
16 }
```

Prever

Com esse código, os LEDs irão acender na ordem: verde → amarelo → vermelho?

- ☐ Sim
- ☐ Não

Observar e explicar

Após executar a simulação: Descreva a função de cada LED no ciclo do semáforo e explique como a temporização dada por `delay()` determina tanto a ordem quanto a duração de cada fase (verde, amarelo e vermelho).

Questão B – Alteração dos Tempos***Instrução***

Modifique os tempos de cada fase (por exemplo, reduzir o tempo do verde ou aumentar o do vermelho) e observe o comportamento resultante.

Prever

Ao alterar os tempos dos `delay()`, o ritmo do semáforo irá mudar?

- ☐ Sim
- ☐ Não

Observar e explicar

Após executar a simulação com novos tempos: Descreva como a alteração dos tempos modificou o fluxo visual do semáforo. Explique também como o tempo de cada LED influencia a percepção e o entendimento do ciclo completo pelo usuário.

Questão C – Falha Simulada (LED Amarelo Apagado)

Código sugerido:

Código-fonte 14 – Simulação de falha: LED amarelo desativado.

```
1 void setup() {  
2   pinMode(3, OUTPUT);  
3   pinMode(4, OUTPUT); // amarelo (não acende)  
4   pinMode(5, OUTPUT);  
5 }  
6  
7 void loop() {  
8   digitalWrite(3, HIGH); delay(4000);  
9   digitalWrite(3, LOW);  
10  
11   // LED amarelo omitido propositalmente (falha)  
12   delay(1500);  
13  
14   digitalWrite(5, HIGH); delay(4000);  
15   digitalWrite(5, LOW);  
16 }
```

Prever

Com o LED amarelo desligado, o ciclo do semáforo ficará incompleto?

- ☐ Sim
- ☐ Não

Observar e explicar

Após simular: Descreva o comportamento do semáforo sem o LED amarelo e explique como a ausência dessa etapa afeta a segurança e o entendimento do ciclo (e.g., transição direta de verde → vermelho).